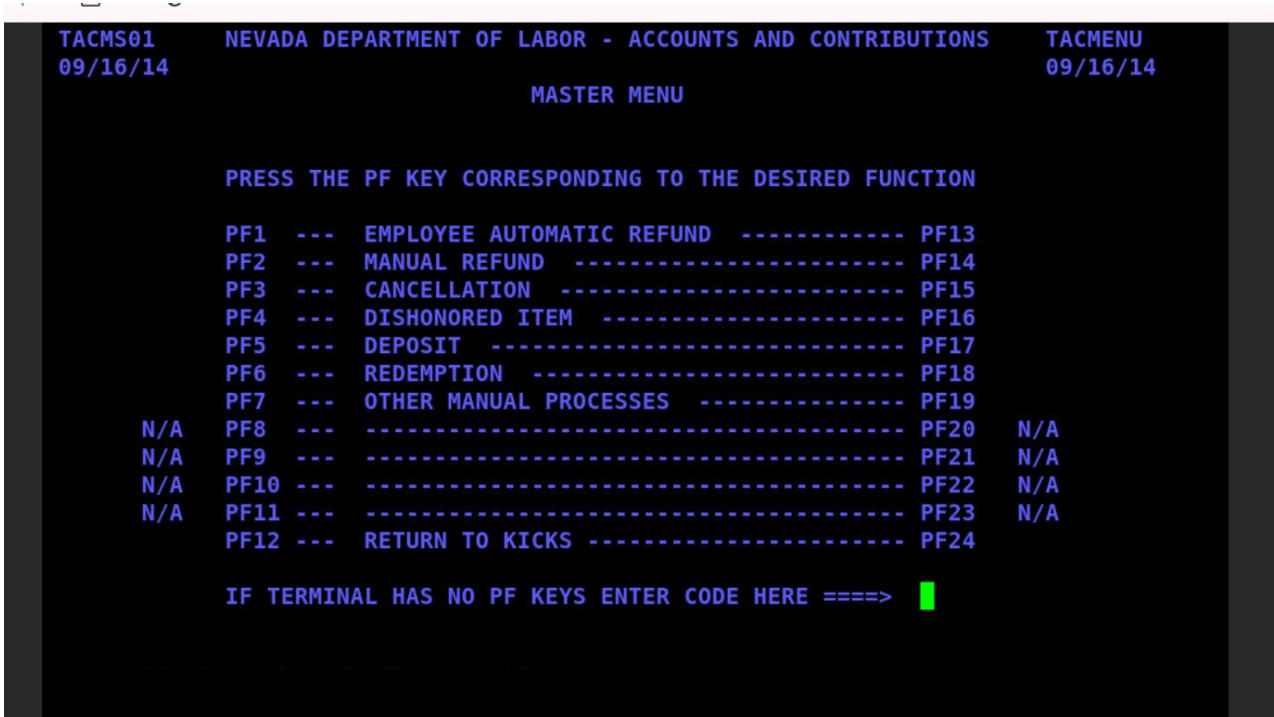


INSTALLATION DE KICKS 1.5.0

sur TK5 / MVS 3.8J dans Docker

Guide pas à pas, commandes vérifiées, erreurs rencontrées et validation transactionnelle

Lab	~/oculix-mainframe-lab / target-mainframe-kicks
Validation	KICKS + transaction BTC0/TAC opérationnelle le 27/08/2026



Validation finale : le menu transactionnel TAC est accessible via BTC0.

STATUT Installation validée de bout en bout : import XMI, restauration des PDS, personnalisation KFIX, création des VSAM, démarrage KICKS et ouverture du menu métier BTC0.

0. Objet, périmètre et conventions

Ce document consolide le chemin d'installation réellement exécuté dans le lab, en supprimant les fausses pistes et en conservant les corrections qui ont été nécessaires. Il vise une reproduction fiable sur le même socle : TK5/MVS 3.8J sous Hercules, dans le container Docker target-mainframe-kicks, avec KICKS 1.5.0.

PÉRIMÈTRE KICKS est ici utilisé comme environnement transactionnel de type CICS dans TSO. Le but est un lab 3270 reproductible pour automatisation visuelle et tests, pas une reproduction complète d'un z/OS/CICS moderne.

Les quatre contextes de commande

Contexte	Repère	Utilisé pour	Exemple
WSL / shell hôte	julien@DESKTOP...\$	Docker, logs, soumission au reader socket	docker exec ...
Hercules Web console	http://localhost:8039	Commandes Hercules et opérateur MVS	devinit 10c ... / r 00, cancel
TSO READY / noVNC	READY	EXEC, LISTDS, ISPF	EXEC KICKSSYS...
ISPF / REVEDIT	Command ===>	Editer et soumettre les membres JCL	C PUB002 WORK01 ALL / SUBMIT

RÈGLE DE SÉCURITÉ OPÉRATOIRE Dans cette TK5, les commandes opérateur MVS envoyées depuis la Hercules Web console doivent être préfixées par "/". Exemple correct : /r 00, cancel. Ne pas utiliser .r.

Sommaire

- 1. Architecture Docker et points d'accès
- 2. Préparer le XMI et les lecteurs de cartes
- 3. Copier le XMI vers un dataset MVS
- 4. Recevoir BIGPDS et restaurer les datasets KICKS
- 5. Personnaliser l'installation avec KFIX
- 6. Charger les données VSAM : MURACH, TAC, SDB, INTRA, TEMP
- 7. Démarrer KICKS et vérifier le runtime
- 8. Valider la transaction métier BTC0
- 9. Diagnostic des erreurs rencontrées
- 10. Persistance du container et checklist finale
- Annexe A. Séquence de commandes compacte
- Annexe B. Sources officielles

1. Architecture Docker et points d'accès

Second environnement TK5 dédié à KICKS, isolé du container mainframe initial.

Le service utilisé pendant l'installation est target-mainframe-kicks, construit depuis ./target-mainframe-kicks. Les mappings de ports validés sont :

Service	Port hôte	Port container / usage
3270	3271	3270 - TN3270
VNC	5901	5900 - VNC
noVNC	6081	6080 - navigateur
Hercules Web	8039	8038 - console Hercules

Construction et démarrage du service

```
cd ~/oculix-mainframe-lab
docker compose build target-mainframe-kicks
docker compose up -d target-mainframe-kicks
```

ACCÈS noVNC : <http://localhost:6081/vnc.html> | Hercules Web console : <http://localhost:8039> | TN3270 direct : localhost:3271

```
julien@DESKTOP-KJSP6MA: ~/oculix-mainframe-lab
hostname: target-mainframe
mem_limit: "1073741824"
networks:
  lab-net: null
ports:
  - mode: ingress
    target: 3270
    published: "3270"
    protocol: tcp
  - mode: ingress
    target: 5900
    published: "5900"
    protocol: tcp
  - mode: ingress
    target: 6080
    published: "6080"
    protocol: tcp
restart: unless-stopped
target-mainframe-kicks:
  build:
    context: /home/julien/oculix-mainframe-lab/target-mainframe-kicks
    dockerfile: Dockerfile
cpus: 1
container_name: target-mainframe-kicks
hostname: target-mainframe-kicks
mem_limit: "1073741824"
networks:
  lab-net: null
ports:
  - mode: ingress
    target: 3270
    published: "3271"
    protocol: tcp
  - mode: ingress
    target: 5900
    published: "5901"
    protocol: tcp
  - mode: ingress
    target: 6080
    published: "6081"
    protocol: tcp
restart: unless-stopped
networks:
  lab-net:
    name: oculix-mainframe-lab_lab-net
    driver: bridge
julien@DESKTOP-KJSP6MA: ~/oculix-mainframe-lab$ docker compose build target-mainframe-kicks
[+] Building 11.0s (4/13)
=> [target-mainframe-kicks internal] load build definition from Dockerfile
=> [target-mainframe-kicks internal] load metadata for docker.io/library/debian:trixie-slim
=> [target-mainframe-kicks internal] load .dockerignore
=> [target-mainframe-kicks internal] transferring context
=> [target-mainframe-kicks 1/9] FROM docker.io/library/debian:trixie-slim@sha256:d7e12182ce18b85b93007c1dedf31f2d29e01ccf3182cc4017c709b6259bc132
=> resolve docker.io/library/debian:trixie-slim@sha256:d7e12182ce18b85b93007c1dedf31f2d29e01ccf3182cc4017c709b6259bc132
=> sha256:d7e12182ce18b85b93007c1dedf31f2d29e01ccf3182cc4017c709b6259bc132 8.97kB / 8.97kB
=> sha256:abc9c8885587630d7f915f27b230c668e250f8f6c457aa4452b534c1bbf73f 1.02kB / 1.02kB
=> sha256:16310eb16bf4251731fab01e8f633bf5e2d75a657ccad97f420b1f83cce457be 14.68MB / 29.79MB
=> [target-mainframe-kicks internal] load build context
=> transferring context: 3.84kB
```

Construction du second service `target-mainframe-kicks` et mappings de ports observés dans le lab.

Stockage DASD utilisé

Le volume attendu par les JCL KICKS est PUB002, absent dans TK5. Le volume de travail choisi et validé est WORK01 (device 0290, 3390).

Hercules Web console - commande opérateur MVS

```
/D U, DASD, ONLINE
```

CHOIX VALIDÉ Remplacer PUB002 par WORK01 dans les DEFINE VSAM des jobs d'installation.

Compte TSO

```
USERID : HERC01
PASSWORD : CUL8TR
```

NOTE HERC01/CUL8TR est le compte public standard de cette distribution TK5. Ne pas réutiliser ce couple sur un autre environnement.

2. Préparer le XMI et les lecteurs de cartes

Le fichier `kicks-tso-v1r5m0.xmi` doit être visible dans `/opt/tk5/jcl` du container.

Source utilisée : dépôt GitHub [moshix/kicks](https://github.com/moshix/kicks), archive `kicks-tso-v1r5m0.zip`. Le fichier final doit se trouver ici dans le container :

```
/opt/tk5/jcl/kicks-tso-v1r5m0.xmi
```

Si le XMI est déjà extrait sur l'hôte, la copie dans le container se fait ainsi :

```
docker cp /chemin/vers/kicks-tso-v1r5m0.xmi target-mainframe-kicks:/opt/tk5/jcl/kicks-tso-v1r5m0.xmi
```

Rôle des deux readers

- 00C : reader socket exposé sur le port 3505 ; il reçoit les JCL envoyés depuis WSL.
- 10C : reader local utilisé par le job IEBGENER pour lire physiquement le flux XMI monté par devinit.

Hercules Web console

```
devlist 00c
devlist 10c
```

Monter ensuite le XMI sur le device 10C :

Hercules Web console

```
devinit 10c /opt/tk5/jcl/kicks-tso-v1r5m0.xmi ebcdic eof
devlist 10c
```

RÉSULTAT ATTENDU HHC02245I ... device initialized. Après lecture complète, devlist 10c doit finir sur eof.

3. Copier le XMI vers un dataset MVS

Soumission du job getfromcardreader par le reader socket 3505.

La version finale du job doit contenir le USER/PASSWORD dans la JOB card et utiliser WORK01, sinon RAKF bloque l'accès au HLQ HERC01 ou MVS attend un volume PUB002 inexistant.

WSL - job getfromcardreader

```
docker exec -i target-mainframe-kicks bash -c \
'cat > /dev/tcp/127.0.0.1/3505' <<'EOF'
//HERC01G JOB CLASS=A,MSGCLASS=H,MSGLEVEL=(1,1),
//          REGION=3M,USER=HERC01,PASSWORD=CUL8TR
//SCRATCH EXEC PGM=IEFBR14
//SYSUT2 DD DSN=HERC01.KICKS.V1R5M0.XMI,DISP=(MOD,DELETE),
//          UNIT=SYSDA,SPACE=(TRK,(0))
//LOAD EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSIN DD DUMMY,DCB=BLKSIZE=80
//SYSUT1 DD UNIT=10C,DISP=OLD,DCB=(RECFM=FB,LRECL=80,BLKSIZE=3200)
//SYSUT2 DD DSN=HERC01.KICKS.V1R5M0.XMI,DISP=(,CATLG),
//          DCB=(DSORG=PS,RECFM=FB,LRECL=80,BLKSIZE=3200),
//          UNIT=SYSDA,VOL=SER=WORK01,SPACE=(TRK,(225,15),RLSE)
EOF
```

VALIDATION Le job HERC01G doit terminer avec SCRATCH RC=0000 et LOAD / IEBGENER RC=0000.

Contrôle côté WSL

```
docker exec target-mainframe-kicks sh -lc \
"grep -E 'HERC01G|IEBGENER|RC=' /opt/tk5/log/hardcopy.log | tail -30"
```

4. Recevoir BIGPDS et restaurer les datasets KICKS

Deuxième job RECV370, puis soumission du membre V1R5M0 depuis TSO.

WSL - job receivexmi

```
docker exec -i target-mainframe-kicks bash -c \
'cat > /dev/tcp/127.0.0.1/3505' <<'EOF'
//HERC01R JOB CLASS=A,MSGCLASS=H,MSGLEVEL=(1,1),
//          REGION=4096K,USER=HERC01,PASSWORD=CUL8TR
//RECV370 EXEC PGM=RECV370
//RECVLOG DD SYSOUT=*
//XMITIN DD DSN=HERC01.KICKS.V1R5M0.XMI,DISP=SHR
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DSN=&&SYSUT1,
//          UNIT=SYSDA,
//          SPACE=(TRK,(300,60)),
//          DISP=(NEW,DELETE,DELETE)
//SYSUT2 DD DSN=HERC01.KICKS.V1R5M0.BIGPDS,
//          UNIT=(SYSDA,SEP=SYSUT1),
//          SPACE=(TRK,(300,60,20)),
//          DISP=(NEW,CATLG,DELETE)
//SYSIN DD DUMMY
EOF
```

VALIDATION RECV370 doit terminer en RC=0000 et créer HERC01.KICKS.V1R5M0.BIGPDS.

Vérifier les membres depuis TSO

TSO READY

```
LISTDS 'HERC01.KICKS.V1R5M0.BIGPDS' MEMBERS
```

Le PDS contient notamment V1R5M0, HCB2, HCOB, HINSTLIB, HKIKRPL, HMAPSRC, SCMDPROC, SCOB, SINSTLIB, SKIKRPL, SMACLIB, SMAPSRC, SPROCLIB, STSTFIL, etc.

```

                                ISPF primary option menu
Option  ==> █

0  ISPF PARMS    Specify terminal and user parms
1  BROWSE        Display source data using Review
2  EDIT          Change source data using Revedit
R  RPF           Browse, EDIT, Reset, PDS with RPF
3  UTILITIES     Perform utility functions
4  FOREGROUND    Invoke language processors
5  BATCH         Submit job for language processing
6  COMMAND       Enter TSO command or CLIST
7  DIALOG TEST   Perform dialog testing
C  CHANGES      Summary of changes for this release
M  TSOAPPLS      Productivity tools and handy apps
T  TUTORIAL      Display information about ISPF
T1 TK5           Summary of changes made in TK5
X  EXIT          Terminate ISPF using log and list defaults

USERID   : HERC01
TIME     : 14:57
TERMINAL : 3277
PF KEYS  : 0012
PANEL    : ISP@PRIM
SCREEN    : 1
LANGUAGE : ENGLISH
APPL ID   : ISP
RELEASE   : V2.2.000
PRIM      : YES
SYSTEMID  : TK5R
PROC      : ISPLGON

Enter END command to terminate ISPF.
```

ISPF sur HERC01/TK5 : point d'entrée pour éditer les membres restaurés.

Soumettre le batch de restauration

Dans ISPF option 2, ouvrir le membre fully-qualified entre apostrophes :

```
'HERC01.KICKS.V1R5M0.BIGPDS(V1R5M0)'
```

REVEDIT - champ Command ==>

```
SUBMIT
```

POURQUOI LES APOSTROPHES Sans apostrophes, ISPF peut préfixer automatiquement le dataset avec HERC01 et produire un nom du type HERC01.HERC01....

Contrôle global

```
docker exec target-mainframe-kicks sh -lc \
"grep 'RC=' /opt/tk5/log/hardcopy.log | tail -40"
```

RÉSULTAT ATTENDU Tous les membres reçus par le job RCVKICK2 doivent être en RC=0000.

5. Personnaliser l'installation avec KFIX

KFIX adapte les datasets et paramètres KICKS à HERC01 et à la distribution TK5.

TSO READY

```
EXEC KICKSSYS.V1R5M0.CLIST(KFIX)
```

Répondre YES lorsque KFIX confirme l'HLQ HERC01. Le run validé a identifié la distribution comme Turnkey MVS et a appliqué le changement TCP attendu.

Sortie validée

```
LOOKS LIKE TURNKEY MVS
-- WILL CHANGE TCP TO 2$
DONE!
READY
```

CRITÈRE Le message DONE! au-dessus du prochain READY confirme la fin correcte de KFIX.

6. Charger les données VSAM

Ordre validé : LOADMUR, LOADTAC, LOADSDB, LODINTRA, LODTEMP.

6.1 LOADMUR

ISPF option 2 - dataset à ouvrir

```
'HERC01.KICKS.V1R5M0.INSTLIB(LOADMUR)'
```

REVEDIT

```
C PUB002 WORK01 ALL
SUBMIT
```

```
REVEDIT  BOTTOM OF DATA REACHED
Command ==> █
000023  DEFINE CLUSTER
000024      (NAME(HERC01.KICKS.MURACH.CUSTMAS) VOLUMES(WORK01
000025      TRACKS(15 15) INDEXED
000026      SHAREOPTIONS(1 3) UNIQUE
000027      RECORDSIZE(118 118)
000028      KEYS(6 0)
000029      )
000030      DATA ( NAME(HERC01.KICKS.MURACH.CUSTMAS.DATA) )
000031      INDEX ( NAME(HERC01.KICKS.MURACH.CUSTMAS.INDEX) )
000032  DEFINE CLUSTER
000033      (NAME(HERC01.KICKS.MURACH.PRODUCT) VOLUMES(WORK01
000034      TRACKS(15 15) INDEXED
000035      SHAREOPTIONS(1 3) UNIQUE
000036      /* RECORDSIZE(39 39)      REC W/COMP-3  */
000037      RECORDSIZE(46 46)
000038      KEYS(10 0)
000039      )
000040      DATA ( NAME(HERC01.KICKS.MURACH.PRODUCT.DATA) )
000041      INDEX ( NAME(HERC01.KICKS.MURACH.PRODUCT.INDEX) )
000042  DEFINE CLUSTER
000043      (NAME(HERC01.KICKS.MURACH.INVOICE) VOLUMES(WORK01
000044      TRACKS(15 15) INDEXED
```

LOADMUR modifié pour allouer les clusters VSAM sur WORK01.

Contrôle

```
docker exec target-mainframe-kicks sh -lc \
"grep -E 'MURACH|IDCAMS|REPRO|RC=|IEF453I' /opt/tk5/log/hardcopy.log | tail -50"
```

RÉSULTAT VALIDÉ DELDEF, LOAD1A/B, LOAD2A/B, LOAD3A/B, LOAD4A/B et BLDIDX : RC=0000.

6.2 LOADTAC - attention au bon membre

ISPF option 2

```
'HERC01.KICKS.V1R5M0.INSTLIB(LOADTAC)'
```

REVEDIT - vérifier avant SUBMIT

```
F TACDATA
C PUB002 WORK01 ALL
```

```

REVEDIT 'TACDATA' FOUND                                COLUMNS 1 72
Command ===>                                           Scroll ===> CSR
***** ZAP*****AUTOSAVE***** TOP OF DATA *****
000001 //TACDATA JOB CLASS=C,MSGCLASS=A,MSGLEVEL=(1,1)
000002 /*
000003 /** LOAD TEST DATA FOR 'TAC' KICKS SAMPLE APPS
000004 /**
000005 /** STKCARDS USED BELOW IS A 'C' PROGRAM TO COPY MULTIPLE
000006 /** SHORT (CARD IMAGE) RECORDS TO A LONGER RECORD. IT'S ALSO
000007 /** USED TO SET LRECL AS NEEDED FOR REPRO.
000008 /**
000009 /** FIRST DELETE/DEFINE TACDATA
000010 /**
000011 //DELDEF EXEC PGM=IDCAMS,REGION=1024K
000012 //SYSPRINT DD SYSOUT=*
000013 //SYSIN DD *
000014 DELETE KICKS.TACDATA
000015 DELETE KICKS.TACDATA
000016 DELETE HERC01.KICKS.TACDATA
000017 SET MAXCC = 0
000018 DEFINE CLUSTER
000019     (NAME(HERC01.KICKS.TACDATA) VOLUMES(PUB002)
000020     TRACKS(15 15) NONINDEXED
000021     SHAREOPTIONS(1 3) UNIQUE

```

Le vrai membre LOADTAC : JOB TACDATA et DEFINE du cluster HERC01.KICKS.TACDATA.

CONTRÔLE INDISPENSABLE Avant SUBMIT, vérifier que la JOB card commence par //TACDATA JOB. Si JES affiche ensuite MURACH, vous avez resoumis LOADMUR par erreur.

REVEDIT

```
SUBMIT
```

RÉSULTAT VALIDÉ Job TACDATA : DELDEF RC=0000, LOAD1 STKCARDS RC=0000, LOAD2 IDCAMS RC=0000.

6.3 LOADSDB - divergence de chemin et correction syntaxique

CHEMIN VALIDÉ DANS CE LAB Le membre réellement utilisable a été trouvé dans HERC01.KICKS.V1R5M0.INSTLIB(LOADSDB). Dans cette installation restaurée, l'ouverture du chemin KICKSSYS correspondant ne donnait pas le JCL attendu.

ISPF option 2

```
'HERC01.KICKS.V1R5M0.INSTLIB(LOADSDB)'
```

REVEDIT

```
C PUB002 WORK01 ALL
```

Après le remplacement, inspecter la ligne DEFINE. Elle doit contenir exactement :

```
VOLUMES(WORK01) -
CYLINDERS(10 10) INDEXED -
```

PIÈGE RENCONTRÉ Le premier run contenait VOLUMES(WORK01 - sans parenthèse fermante. IDCAMS a alors renvoyé IDC3201I CONSTANT CYLINDERS EXCEEDS LENGTH LIMIT et RC=0012.

REEDIT après correction

```
SUBMIT
```

RÉSULTAT VALIDÉ Job LOADSDB corrigé : DELDEF RC=0000 et LOAD RC=0000.

6.4 LODINTRA

ISPF option 2

```
'HERC01.KICKSSYS.V1R5M0.INSTLIB(LODINTRA) '
C PUB002 WORK01 ALL
SUBMIT
```

RÉSULTAT VALIDÉ LODINTRA : DELDEF RC=0000, LOAD RC=0000.

6.5 LODTEMP

ISPF option 2

```
'HERC01.KICKSSYS.V1R5M0.INSTLIB(LODTEMP) '
C PUB002 WORK01 ALL
SUBMIT
```

RÉSULTAT VALIDÉ LODTEMP : DELDEF RC=0000, LOAD RC=0000.

6.6 Contrôle synthétique des jobs

```
docker exec target-mainframe-kicks sh -lc \
"tail -80 /opt/tk5/log/hardcopy.log"
```

Etat final validé

LOADSDB	DELDEF	IDCAMS	RC=0000
LOADSDB	LOAD	IDCAMS	RC=0000
LODINTRA	DELDEF	IDCAMS	RC=0000
LODINTRA	LOAD	IDCAMS	RC=0000
LODTEMP	DELDEF	IDCAMS	RC=0000
LODTEMP	LOAD	IDCAMS	RC=0000
TACDATA	DELDEF	IDCAMS	RC=0000
TACDATA	LOAD1	STKCARDS	RC=0000
TACDATA	LOAD2	IDCAMS	RC=0000

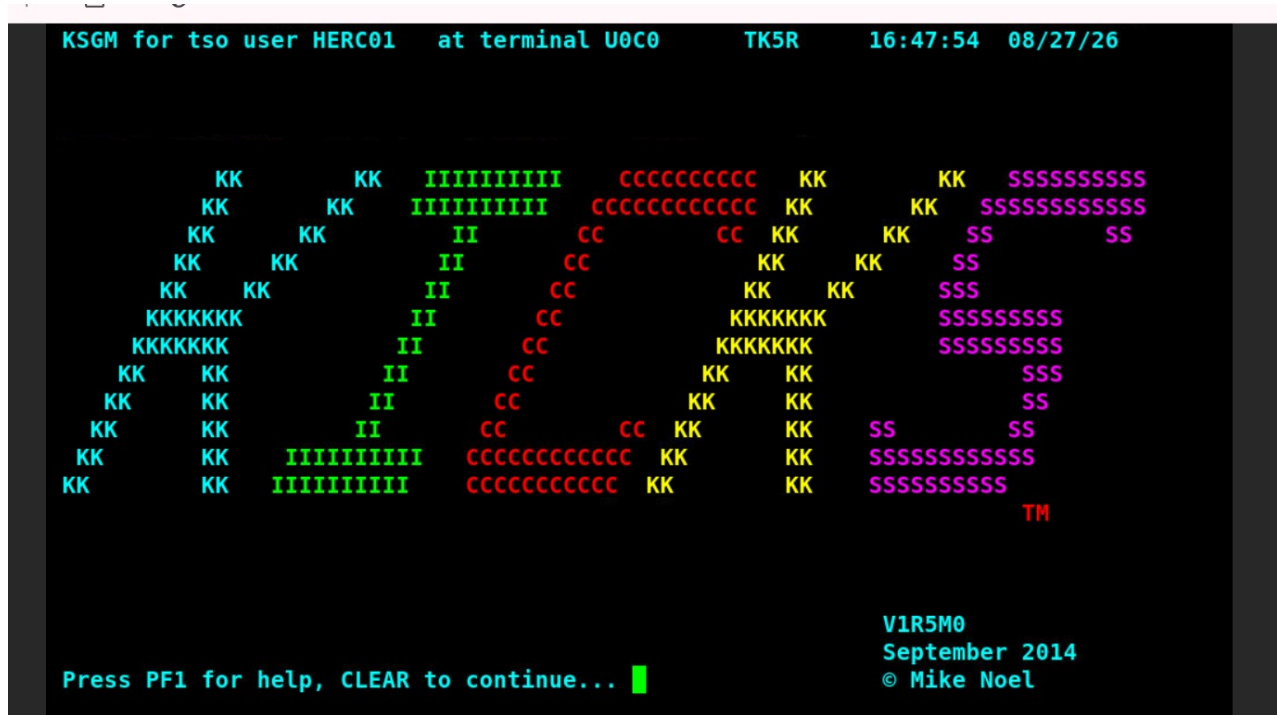
7. Démarrer KICKS et vérifier le runtime

Le démarrage se fait depuis le prompt TSO READY.

TSO READY

```
EXEC KICKSSYS.V1R5M0.CLIST(KICKS)
```

Le startup charge les modules KIKSIT1\$, KIKPCP1\$, KIKKCP1\$, KIKFCP1\$, KIKDCP1\$, KIKBMS1\$, KIKTCP2\$, KIKSCP1\$, KIKTSP1\$, KIKPPT1\$, KIKPCT1\$, KIKFCT1\$ et KIKDCT1\$.



Écran KSGM après correction de TACDATA : KICKS 1.5.0 est lancé pour HERC01.

VALIDATION RUNTIME L'écran KSGM "Press PF1 for help, CLEAR to continue..." confirme que KICKS est actif dans la session TSO.

Dans c3270 via noVNC, la touche CLEAR a été obtenue avec Ctrl+C. Après CLEAR, l'écran devient vierge et accepte un code transaction.

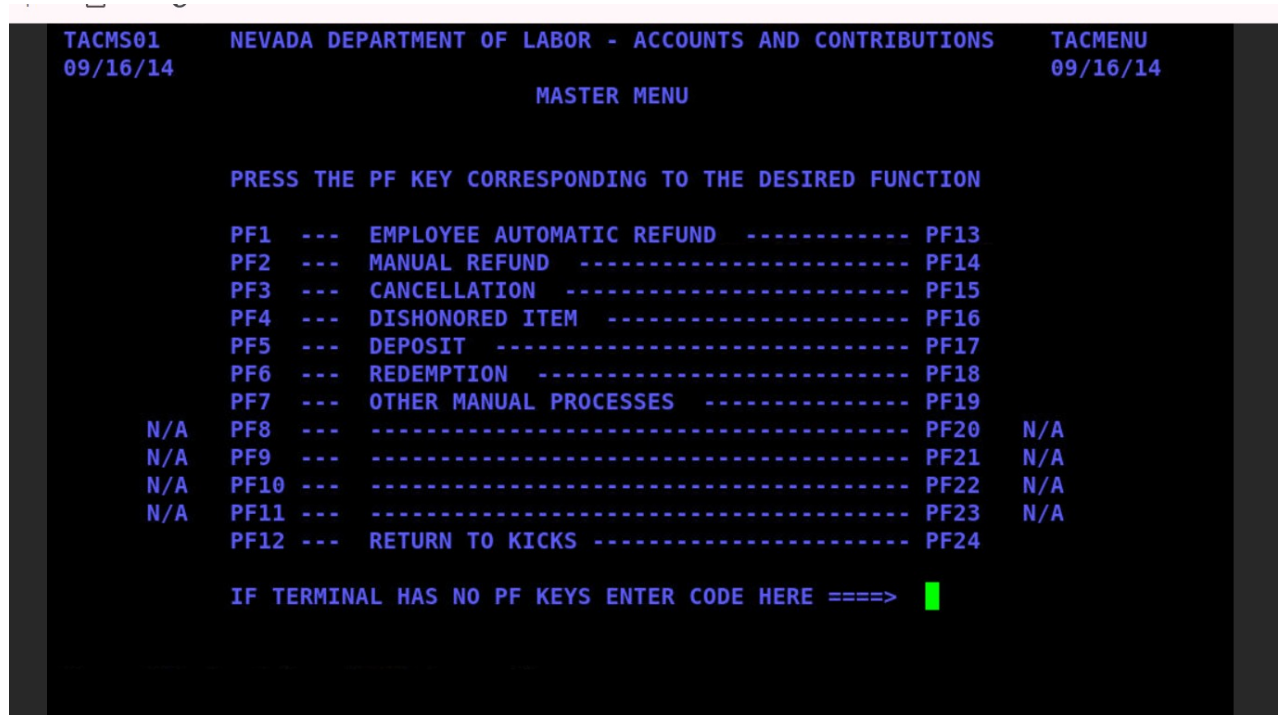
Ctrl+C # CLEAR dans c3270/noVNC

8. Valider la transaction métier BTC0

Cette étape prouve que TACDATA est non seulement créé mais réellement exploitable par une transaction.

Depuis l'écran KICKS vierge

BTC0
<Entrée>



Menu TAC obtenu par BTC0 : Nevada Department of Labor - Accounts and Contributions.

PREUVE FONCTIONNELLE Le MASTER MENU TAC s'affiche avec PF1 Employee Automatic Refund, PF2 Manual Refund, PF3 Cancellation, PF4 Dishonored Item, PF5 Deposit, PF6 Redemption et PF7 Other Manual Processes.

Sortie propre de KICKS

```

Ctrl+C
KSSF
<Entrée>

```

KSSF est la transaction de signoff. Après arrêt, la session revient vers TSO.

9. Diagnostic des erreurs rencontrées

Ces erreurs sont conservées ici parce qu'elles évitent de perdre du temps lors d'une réinstallation.

9.1 PUB002 absent / allocation suspendue

```
IEF238D ... REPLY DEVICE NAME OR 'CANCEL'
```

Cause : JCL standard KICKS ciblant PUB002, absent de TK5. Correction : WORK01. Pour annuler un job suspendu depuis la Hercules Web console :

```

/r 00,cancel
/r 01,cancel

```

9.2 RAKF refuse le job soumis par reader

```

RAKF0005 INVALID ATTEMPT TO ACCESS RESOURCE
RAKF000A PROD ,HERC01G ,DATASET ,HERC01.KICKS.V1R5M0.

```

Cause : le job envoyé par le reader socket s'exécutait sous PROD. Correction : ajouter USER=HERC01,PASSWORD=CUL8TR dans la JOB card.

9.3 LOADSDB RC=0012

```

IDC3201I CONSTANT 'CYLINDERS' EXCEEDS LENGTH LIMIT
IDC3202I ABOVE TEXT BYPASSED UNTIL NEXT COMMAND. CONDITION CODE IS 12

```

Cause : parenthèse fermante absente après WORK01. Ligne correcte : VOLUMES(WORK01) -.

9.4 TACDATA désactivé au startup

```

READY
EXEC KICKSSYS.V1R5M0.CLIST(KICKS)
KICKS version 1.5.0(0) startup using SIT=1$,
  which with command line and stdin overrides results in..
loading KIKSIT1$ - 80 bytes at CE130, ep CE130
loading KIKPCP1$ - 59600 bytes at 14E730, ep 14E7B8
loading KIKKCP1$ - 22216 bytes at 15D938, ep 15E168
loading KIKFCP1$ - 40280 bytes at 1632A8, ep 16357C
loading KIKDCP1$ - 35616 bytes at 16D4E0, ep 16D654
loading KIKBMS1$ - 17072 bytes at 176D50, ep 177278
loading KIKTCP2$ - 28360 bytes at 17B138, ep 17B228
loading KIKSCP1$ - 3456 bytes at B5280, ep B5290
loading KIKTSP1$ - 15072 bytes at 182520, ep 182538
loading KIKPPT1$ - 4080 bytes at B6010, ep B6010
loading KIKPCT1$ - 1464 bytes at 14E178, ep 14E178
loading KIKFCT1$ - 9512 bytes at 186AD8, ep 186AD8
loading KIKDCT1$ - 256 bytes at CE030, ep CE030

OPID=001, DMPCLASS=A, ICVR=5000, ENQSCOP=SYSTEMS, CWAL=100
TRCFLAGS=1, TCTUAL=100, TRCNUM=100, PLTPI=KSGM, PLTSD=K999
NATLANG=E, MAXDELY=180, FFREEKB=NO

no alloc/DD for file TACDATA , disabling...
*** █

```

Premier démarrage incomplet : "no alloc/DD for file TACDATA, disabling..."

Cause réelle dans ce run : le vrai LOADTAC n'avait pas été exécuté ; un LOADMUR avait été resoumis par erreur.
Correction : ouvrir LOADTAC, vérifier F TACDATA et la JOB card //TACDATA, puis soumettre et obtenir tous les RC à zéro.

9.5 Mauvais code transaction : TAC



Saisie de TAC : Transaction Abend Code APCT.

Le code correct pour ouvrir le menu TAC de l'exemple est BTC0. TAC n'est pas la transaction d'entrée et provoque l'abend APCT.

9.6 Mauvais champ dans REVEDIT

```

REVEDIT  HERC01.KICKS.V1R5M0.INSTLIB(LOADTAC) - 1.51          COLUMNS 1 72
Command ==> Scroll ==> CSR
***** ZAP*****AUTOSAVE***** TOP OF DATA *****
000001 //C PUB002 WORK01 ALLC,MSGCLASS=A,MSGLEVEL=(1,1)
000002 //*
000003 //* LOAD TEST DATA FOR 'TAC' KICKS SAMPLE APPS
000004 //*
000005 //* STKCARDS USED BELOW IS A 'C' PROGRAM TO COPY MULTIPLE
000006 //* SHORT (CARD IMAGE) RECORDS TO A LONGER RECORD. IT'S ALSO
000007 //* USED TO SET LRECL AS NEEDED FOR REPRO.
000008 //*
000009 //* FIRST DELETE/DEFINE TACDATA
000010 //*
000011 //DELDEF EXEC PGM=IDCAMS,REGION=1024K
000012 //SYSPRINT DD SYSOUT=*
000013 //SYSIN DD *
000014 DELETE KICKS.TACDATA
000015 DELETE KICKS.TACDATA
000016 DELETE HERC01.KICKS.TACDATA
000017 SET MAXCC = 0
000018 DEFINE CLUSTER
000019     (NAME(HERC01.KICKS.TACDATA) VOLUMES(PUB002)
000020     TRACKS(15 15) NONINDEXED
000021     SHAREOPTIONS(1 3) UNIQUE

```

Exemple de fausse manipulation : la commande C PUB002 WORK01 ALL a été saisie dans la ligne JCL au lieu du champ Command.

Si cela arrive, restaurer la JOB card puis saisir les commandes REVEDIT uniquement dans le champ Command ==>. Pour LOADTAC, la ligne 1 correcte est :

```
//TACDATA JOB CLASS=C,MSGCLASS=A,MSGLEVEL=(1,1)
```

9.7 Commandes opérateur Hercules

```
Incorrect : .r 00,cancel
Correct   : /r 00,cancel
```

Dans ce TK5, le préfixe / est nécessaire pour transmettre la commande au guest MVS depuis la Web console Hercules.

10. Persistance et checklist finale

Le lab fonctionne ; il faut maintenant éviter de perdre les DASD modifiés.

IMPORTANT Dans la configuration observée, /opt/tk5/dasd n'est pas monté sur un volume Docker externe. Un simple stop/start conserve les modifications, mais un force-recreate, une suppression du container ou une reconstruction peut les perdre.

Créer immédiatement une image snapshot du container installé :

```
docker commit target-mainframe-kicks oculix-mainframe-kicks:1.5.0-installed
```

Éviter ensuite :

```
docker compose up --force-recreate
```

tant que la persistance DASD n'a pas été explicitement externalisée.

Checklist de validation

- ☐ Container target-mainframe-kicks lancé et healthcheck OK.
- ☐ noVNC accessible sur localhost:6081 et Hercules Web sur localhost:8039.
- ☐ Volume WORK01 online et utilisé pour les VSAM KICKS.
- ☐ HERC01.KICKS.V1R5M0.XMI créé depuis le reader 10C.
- ☐ RECV370 et RCVKICK2 terminés en RC=0000.
- ☐ KFIX terminé avec DONE!.
- ☐ LOADMUR terminé en RC=0000.
- ☐ TACDATA terminé en RC=0000.
- ☐ LOADSDB corrigé puis terminé en RC=0000.
- ☐ LODINTRA terminé en RC=0000.
- ☐ LODTEMP terminé en RC=0000.
- ☐ KICKS démarre sans "no alloc/DD for file TACDATA".
- ☐ BTC0 affiche le MASTER MENU TAC.
- ☐ Snapshot Docker créé ou DASD externalisés avant tout recreate.

Annexe A. Séquence compacte des commandes

Cheat sheet pour refaire l'installation sans relire tout le diagnostic.

```
# WSL
cd ~/oculix-mainframe-lab
docker compose build target-mainframe-kicks
docker compose up -d target-mainframe-kicks

# Hercules Web console
/D U,DASD,ONLINE
devlist 00c
devlist 10c
devinit 10c /opt/tk5/jcl/kicks-tso-v1r5m0.xmi ebcdic eof
devlist 10c

# WSL - getfromcardreader
# -> soumettre le JCL HERC01G avec USER=HERC01,PASSWORD=CUL8TR
# et VOL=SER=WORK01 via /dev/tcp/127.0.0.1/3505

# WSL - receivexmi
# -> soumettre HERC01R / RECV370 via le même reader socket

# TSO READY
LISTDS 'HERC01.KICKS.V1R5M0.BIGPDS' MEMBERS

# ISPF option 2
'HERC01.KICKS.V1R5M0.BIGPDS(V1R5M0)'
SUBMIT

# TSO READY
EXEC KICKSSYS.V1R5M0.CLIST(KFIX)

# ISPF / REEDIT
'HERC01.KICKS.V1R5M0.INSTLIB(LOADMUR)'
C PUB002 WORK01 ALL
SUBMIT

'HERC01.KICKS.V1R5M0.INSTLIB(LOADTAC)'
F TACDATA
C PUB002 WORK01 ALL
SUBMIT

'HERC01.KICKS.V1R5M0.INSTLIB(LOADSDB)'
C PUB002 WORK01 ALL
# vérifier : VOLUMES(WORK01) -
SUBMIT

'HERC01.KICKSSYS.V1R5M0.INSTLIB(LODINTRA)'
C PUB002 WORK01 ALL
SUBMIT

'HERC01.KICKSSYS.V1R5M0.INSTLIB(LODTEMP)'
C PUB002 WORK01 ALL
SUBMIT

# TSO READY
EXEC KICKSSYS.V1R5M0.CLIST(KICKS)

# c3270/noVNC
Ctrl+C
BTC0
<Entrée>

# sortie KICKS
Ctrl+C
KSSF
<Entrée>

# WSL - snapshot
docker commit target-mainframe-kicks oculix-mainframe-kicks:1.5.0-installed
```

Annexe B. Sources officielles

Les instructions ont été recoupées avec le dépôt KICKS, puis corrigées selon les résultats réellement observés dans le lab.

Dépôt KICKS: <https://github.com/moshix/kicks>

README: <https://github.com/moshix/kicks/blob/master/README.md>

Guide d'installation KICKS 1.5.0:

<https://github.com/moshix/kicks/blob/master/kicks-tso-v1r5m0/User%27s%20Guide/Installation.shtml>

Guide Operation KICKS 1.5.0:

<https://github.com/moshix/kicks/blob/master/kicks-tso-v1r5m0/User%27s%20Guide/Operation.shtml>

Capture de référence du menu TAC:

<https://github.com/moshix/kicks/blob/master/kicks-tso-v1r5m0/User%27s%20Guide/screenshots/12-tac-master.html>

NOTE DE MÉTHODE Lorsque la documentation générique et le comportement de cette distribution divergeaient, le guide retient le chemin réellement validé par exécution et RC=0000 dans le lab.

FIN DU GUIDE - installation KICKS 1.5.0 validée par transaction BTC0